

Padronização de Detecção com Linguagem Sigma

1. Introdução à Linguagem Sigma

A análise de logs em um SOC (Security Operations Center) enfrenta o desafio constante da falta de padronização. O **Sigma** surge como uma linguagem de assinatura genérica, de código aberto, projetada para descrever eventos de log em um formato estruturado (YAML).

Assim como o Snort é utilizado para tráfego de rede e o Yara para análise de arquivos, o Sigma é o padrão de mercado para **arquivos de log**.

2. Objetivos e Casos de Uso

A implementação do Sigma visa resolver problemas críticos na engenharia de detecção:

- **Portabilidade:** Permite que métodos de detecção sejam compartilhados sem dependência de um fornecedor específico (*Vendor Agnostic*).
- **Interoperabilidade:** Facilita o compartilhamento de assinaturas entre comunidades de Inteligência de Ameaças.
- **Customização:** Oferece uma estrutura para escrever regras baseadas em comportamentos maliciosos específicos, adaptáveis a diferentes ambientes.

3. Arquitetura e Processo de Desenvolvimento

O desenvolvimento de uma detecção via Sigma segue um fluxo lógico que garante a conversão da regra para qualquer solução SIEM:

1. **Formato da Regra (YAML):** A descrição genérica do log estruturado.
2. **Sigma Converter:** Scripts (geralmente em Python) que processam a regra e mapeiam os campos para a sintaxe específica do backend alvo.
3. **Consulta Automatizada:** O resultado final (query) pronto para ser executado no SIEM (Elasticsearch, Splunk, Sentinel, etc.) para filtrar alertas durante investigações.

Estrutura e Sintaxe de Regras Sigma

1. Fundamentos do YAML no Sigma

As regras Sigma utilizam o formato **YAML** (*YAML Ain't Markup Language*) pela sua legibilidade e capacidade de serialização. Para garantir a integridade das regras, os seguintes padrões devem ser seguidos:

- **Case-sensitivity:** O YAML diferencia maiúsculas de minúsculas.
- **Indentação:** Deve-se usar espaços (nunca tabulações) para definir a hierarquia.
- **Extensão:** Os arquivos devem ser salvos obrigatoriamente como `.yaml`.

2. Elementos de Sintaxe da Regra

Uma regra Sigma é composta por campos que fornecem metadados e lógica de detecção:

2.1 Metadados e Controle

- **Title e ID:** O título deve ser conciso. O ID é um UUID único para rastreamento.
- **Status:** Define a maturidade (Ex: `stable`, `test`, `experimental`).
- **Related:** Permite vincular regras (derivadas, obsoletas ou renomeadas).
- **Level:** Define a severidade da ameaça (`informational`, `low`, `medium`, `high`, `critical`).

2.2 Logsource (Origem dos Dados)

Define onde a regra deve ser aplicada. Pode ser filtrada por:

- **Product:** (Ex: `windows`, `linux`).
- **Category:** (Ex: `process_creation`, `webserver`).
- **Service:** (Ex: `sysmon`, `sshd`).

3. Lógica de Detecção

O campo `detection` é o núcleo da regra e divide-se em **identificadores de pesquisa e condições**.

3.1 Identificadores: Listas e Mapas

- **Listas (Operação OU):** Identificadores listados com hífen são processados logicamente como "OU". Se qualquer valor coincidir, a seleção é positiva.
- **Mapas (Operação E):** Pares de chave/valor onde todos os elementos devem ser verdadeiros simultaneamente para validar a detecção.

3.2 Modificadores de Valor

Anexados ao campo via barra vertical (`|`), alteram como o valor é processado:

- **Transformação:** `contains`, `endswith`, `startswith`, `base64`.
- **Tipo:** `re` (expressão regular, comum para consultas em Elasticsearch).

4. Expressões Condicionais

A condition define a lógica final da regra usando operadores como and, or, not e seletores de quantidade (1 of..., all of...).

Exemplo de Lógica de Persistência (WMI): A regra pode buscar a criação de assinaturas de eventos (EventIDs 19, 20, 21) e filtrar atividades legítimas de ferramentas conhecidas usando condition: selection and not filter.

Desenvolvimento e Operacionalização de Regras Sigma

O desenvolvimento de uma regra Sigma eficaz não é apenas um exercício de sintaxe, mas um processo analítico que traduz inteligência de ameaças (Threat Intel) em lógica de detecção acionável.

1. Ciclo de Desenvolvimento de Detecção (Cenário AnyDesk)

Para ilustrar o processo, analisamos o uso do **AnyDesk** (ferramenta legítima) sendo instalada silenciosamente por adversários para estabelecer Comando e Controle (C2).

1.1 Análise de Inteligência (Intel Analysis)

A partir de relatórios de incidentes (ex: *TheDFIRReport*), identificamos artefatos críticos de instalação:

- **Comandos de Instalação:** Uso de flags específicas como `--install`, `--start-with-win` e `--silent`.
- **Caminhos de Execução:** O executável é frequentemente movido para diretórios como `C:\ProgramData\AnyDesk.exe`.
- **Persistência:** Criação de contas de usuário locais (ex: `oldadministrator`) e manipulação de chaves de registro em `SpecialAccounts`.

1.2 Estruturação da Regra (YAML)

Com base na análise, a regra é construída focando na **Criação de Processos** (`process_creation` no Windows):

YAML

title: AnyDesk Installation

status: experimental

description: Detects AnyDesk Remote Desktop installation often used for unauthorized remote access.

logsource:

```
category: process_creation
product: windows
detection:
  selection:
    CommandLine|contains|all:
      - '--install'
      - '--start-with-win'
    CurrentDirectory|contains:
      - 'C:\ProgramData\AnyDesk.exe'
  condition: selection
falsepositives:
  - Legitimate deployment by IT administration
level: high
tags:
  - attack.command_and_control
  - attack.t1219
```

2. Conversão e Implementação (Sigma Conversion)

As regras Sigma são agnósticas, o que significa que precisam ser traduzidas para a linguagem específica do SIEM em uso (Elasticsearch, Splunk, QRadar, etc.).

2.1 Ferramentas de Conversão

Duas abordagens principais são utilizadas para a tradução:

1. **Sigmac (CLI)**: Ferramenta em Python que utiliza arquivos de configuração (`config.yml`) para mapear campos genéricos do Sigma para os campos específicos do log de destino (ex: Winlogbeat).
 - *Exemplo Splunk*: `sigmac -t splunk -c splunk-windows rule.yml`
2. **Uncoder.io (Web)**: Plataforma online que facilita a tradução rápida entre múltiplos formatos (Sigma para Lucene, KQL, SPL, etc.).

3. Validação e Busca (Threat Hunting)

Após a conversão para a sintaxe do **Elasticsearch (Lucene)**, a consulta é aplicada no **Kibana Discover** para validar a presença de hits no ambiente.

Parâmetros de Validação:

- **Índice**: `winlogbeat-*` ou `filebeat-*`.
- **Filtro Temporal**: Ajustado conforme a janela do incidente suspeito.
- **Objetivo**: Confirmar se os argumentos da linha de comando capturados nos logs coincidem com a lógica definida na regra Sigma.

